Final
C-01
July 2026
Thursday

Kaniz Fatema

8051 - Chp 4 (প্রথম)
- IO Port Programming
Chp 5 : Addressing Mode
Chp 6 : Arithmatic Logical Instructions
Chp 8 : Hardward Connection and Hex File
Chp 9 : Timer Programming (Timer এর 16 mode পড়ব)

8086 - Chp 9 :
Chp 10 :
Chp 11 :
Chp 12 :

8051 mazidi
309 / 677

Chp8
Hardware Connection and Intel Hex File

Intel Hex File

- file formate
- Intel এর সফটওয়্যার executable file তৈরি করার formate
- Assembly device dependent
- Designed to standardize the loading of executable

Source file (.asm)

↓

object File (.obj file)

↓ Linker (multiple obj এর মধ্যে link করে)

abs absolute object file (abs file)

↓ OH utility

Hex file

Obj file এ assembly এর corresponding line এর machine code save হয়।

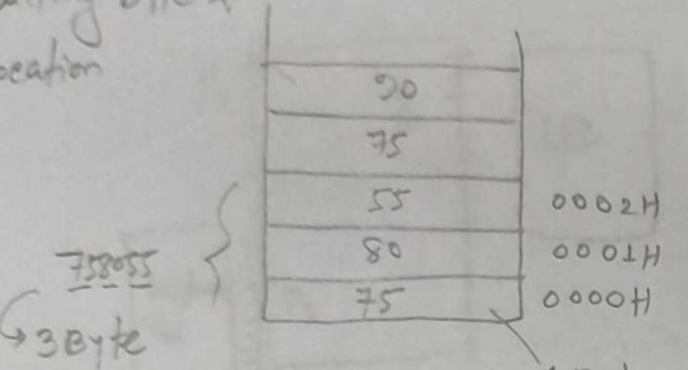
Obj File

→ ROM

Loc	obj	Line	
0000		1	ORG (0H) → ROM এর location
0000	758055	2	MAIN: MOV P0, # 55H
0003	759055	3	MOV P1, # 55H

ROM এর এই value গুলো বা machine code store হবে।

starting byte এর location



32 16 8 4 2 1 0
1

Hex File

machine code থাকলে TT-00 হবে

Count করে byte → 1Byte | starting byte এর location → 2Byte | Termination line কিনা (00 - শেষ লাইন না, 01 - শেষ লাইন) → 1Byte | 16 Byte (32টি D) (machine code | Check sum → 1Byte

: CC AAAA TT DDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDD SS

: 10 0000 00 758055 759055 75A055 7D FA 11 1C 75 80 AA 9F

: 10 0010 00 7590 AA 75 A0 AA 7D FA 11 1C 80E4 7C23 7B4F . 01 35

: 07 0020 00 DB FE DC FA DD F6 22 FF

: 00 0000 01

SS → Direct calculation করা যায় না. এভাবে find

Sum = Sum of Byte

Checksum = 2's compliment Sum

10 + 00 + 00 + 00 + 75 + 80

~~5~~AB, ~~7~~61 ~~4~~FF

0000

0110 0001
1001 1110
+ 1
1001 1111
= 9F

C-02
16 July 2026

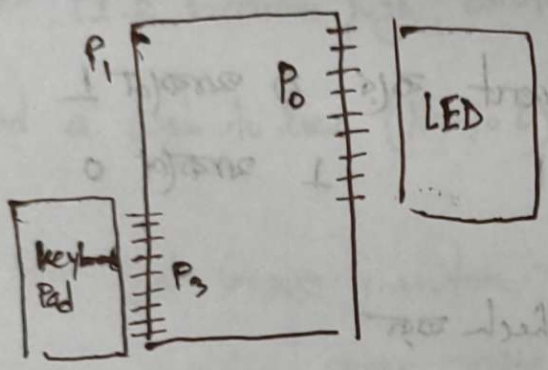
Chapter – 4

133/617

I/O pin → 32 টা
I/O port → 4 টা ↙ 8টা pin এর combination. 8টা pin Parallel ভাবে port হিসাবে কাজ করে।

Mov P1, #50 → o/p → port থেকে output

Mov A, P3 → i/p → Port থেকে input
↓
Register



Port দিয়ে input/output দুই হিসাবেই কাজ করানো যায়। যে ডিভাইস connect থাকবে।

~~কোনো~~ # সবগুলো pin High pass করতে হবে Config করার জন্য

ort for input → P3 কে input হিসাবে config করতে চাইলে এর 8টা Pin কে High দিয়ে Config করব।

0 → output
1 → Input

↳ Reverse for output But mendatory না।

Configuration (i/p) → Alphabet এর আগে Dummy '0' add করতে হয়।

Mov P3, #0FFH
(1111 1111)

Mov A, P3 ← P3 port থেকে Reg A তে value receive হচ্ছে।

Sometimes we need to access only 1 or 2 bits of the port.

1 or single pin এ Data write করা

→ Data write করলেই value 1

SETB P0.0

CLR P0.0 → single pin এর value 0 করে

CPL P0.0 → value invert করে 0 থাকলে 1
1 থাকলে 0

compliment

Pin check in

→ means read + check করা

Pin এর value check in এর জন্য JB (Jump if Bit equals t

JNB (Jump if not Bit equals to

↳ Jump করবে Bit 0 হলে

conditional instruction

JB P1.2, target_label

JNB P1.2, label

Instruction	Function
SETB bit	Set the bit (bit = 1) High করবে always
CLR bit	Clear the bit (bit = 0)
CPL bit	Complement the bit bit toggle করে 0 ⇌ 1
JB bit, target	Jump to target if bit = 1
JNB bit, target	Jump to target if bit = 0
JBC bit, target	Jump to target if bit = 1

P→148/617

Exercise 4.3

Write a program to perform the following

a) Keep monitoring the P1.2 bit until it becomes high.

b) When P1.2 becomes high, write value 45H to port 0

c) Send a high to low (H-to-L) pulse to P2.3

input pin কে কিভাবে monitor করব।

কোন pin কে monitor করতে বলবে সেটা input pin

* Pin কে config করতে High দিতে হবে SetB Port

SETB P1.2 → P1.2 কে High করে input হিসেবে কাজ করব

Again: JB P1.2, High

SJMP Again

High: MOV P0, #45H

SETB P2.3 → High

CLR P2.3 → low

H Pulse

Ex-4.4 Slide (140/619)

```
      SETB P2.3
Again: JB P2.3, High
       SJMP Again
High:  SETB P1.5
       CLR P1.5
       SJMP Again
```

Ex-4.5

```
      SETB P1.7
Again: JB P1.7, Send_Y
       MOV P2, 'N'
       SJMP Again
Send_Y: MOV P2, 'Y'
       SJMP Again
```

-X-

```
      SETB P1.7
      JB P1.7, Send_Y
      JNB P1.7, Send_N
Send_Y: MOV P2, 'Y'
Send_N: MOV P2, 'N'
```

Redundant হয়ে যা
but logic ঠিক
আকারে করা যা

Pin value Read

MOV C, P2.3 → 1 bit

↳ carry-flag (storage place) 1 bit

* যেকোনো pin এর value রিডের জন্য চাইলে carry use করতে.

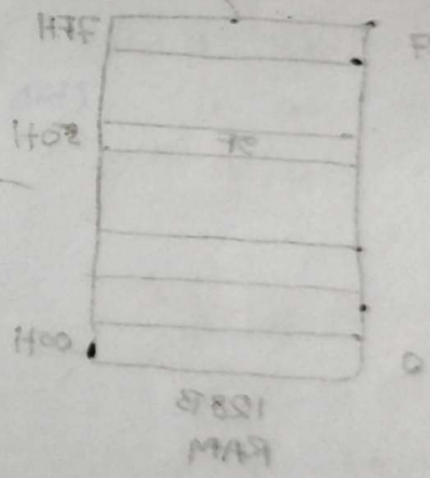
MOV C, P2.3 → Read

MOV P1.7, C → write

JC (Jump if carry)

H.W - (4.6, 4.7)

4.3, 4.4, 4.5 → Clam করতেন



Chapter- 5

Addressing Modes

8051 Registers
↳ A, B, R0—R7

Accessing Memory (128 Byte)

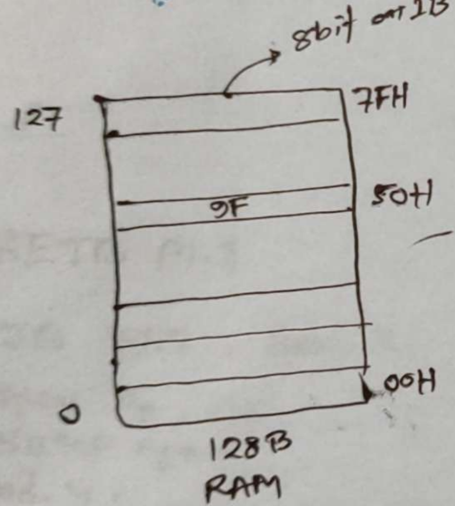
1. Immediate → Mov A, #5
2. Register → Mov A, R0
3. Direct → Mov A, 50H
4. Register Indirect
5 Indexed

↳ শুধু read করব
↳ একবারই write করি

Invalid
Mov R0, R1
↳ দুইটা reg r type হতে পারছে না.

RAM (128 B ~~1028 byte~~)

] Rom (4KB)



Mov A, 50H → memory location বুঝাতে # use করি না
↳ RAM এর 50H location থেকে value read করতেছে

Direct

→ memory location তাই # use করব না

Read: Mov A, 50H

Write: Mov 7FH, A

Mov 127, A

Invalid: memory location এর বাইরে লিখলে Mov 134, A

Mov #127, A

Mov 45, 20 → দুইটাই mem location হবে না

Register Indirect

location এ point করার জন্য R0, R1 use করতে পারব

Mov R0, #50H → mem location এর value R0 এ নিয়ে আসলাম

Mov A, @R0

↳ mem location এর access করার জন্য

Mov R1, #127H

Mov ~~A, @R1~~ @R1, A

Example 5.3

Write a program to copy the value 55H into RAM memory locations 40H to 41H using

a) direct addressing mode
b) register indirect addressing
c) with loop

memory locations
40H
41H
42H

a)
```
Mov A, #55H
Mov 40H, A
Mov 41H, A
Mov 42H, A
```

b)
```
Mov A, #55H
Mov R0, #40H
Mov @R0, A
INC R0
Mov @R0, A
```

with loop

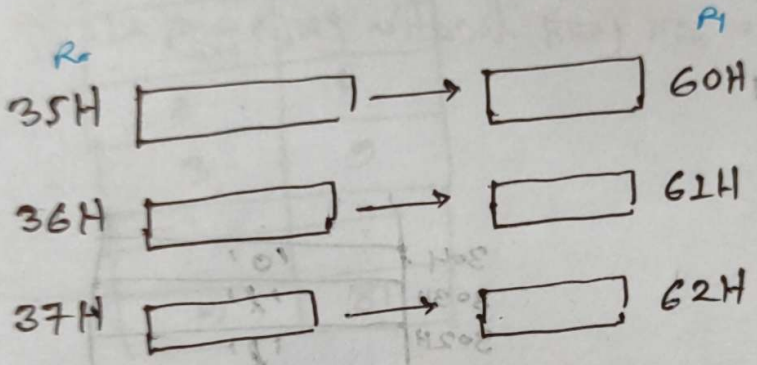
c)
```
Mov A, #55H
Mov R0, #40H
Mov R2, #2
L: Mov @R0, A
   INC R0
   DJNZ R2, L
```

C.05
23-Jul-26

Example 5.4

```
    CLR A
    MOV R0, #60H
    MOV R2, #16
L:  MOV @R0, A
    INC R0
    DJNZ R2, L
```

Example 5.5




```
    CLR A
    MOV R0, #35H
    MOV R1, #60H
    MOV R2, #10H
L:  MOV @R0, A
    MOV A, @R0      read
    MOV @R1, A      write
    INC R0
    INC R1
    DJNZ R2, L
```

179/67

Indexed Addressing Mode

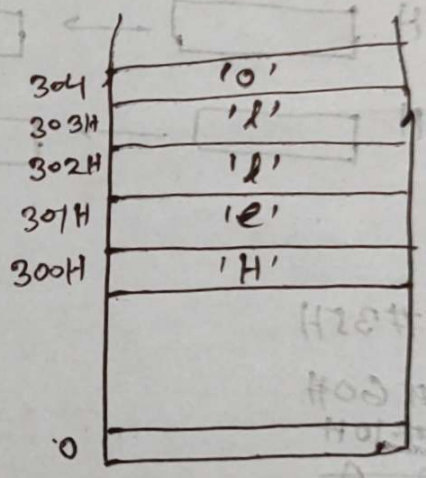
↳ ROM read করার জন্য

Syntax: MOVC A, @A+DPTR [For reading only]

Destination ↳ Data Pointer Register 16bit
- Rom এর location point করতে use হয়

#variable declare করলেই write operation হবে।

```
ORG 00H  → Rom এর কোন location থেকে শুরু হবে
MOV DPTR, #0300H
CLR A
MOV A, @A+DPTR
ORG 300H

DATA: DB "Hello"

DATA2: DB 1,2,3,4

ORG 500H

DATA3

END
```



* variable declare করলে value ROM এ write হবে।

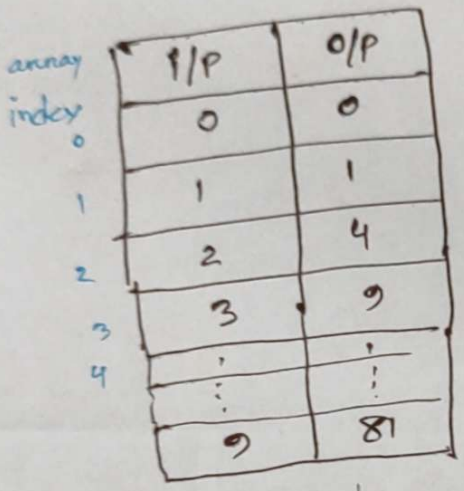
↳ পরে location access Read

```
ORG 00H
MOV R3, #5
MOV DPTR, #0300H
Again: CLR A
       MOVC A, @A+DPTR
       INC DPTR
       DJNZ R3, Again
```

Example-5.8

Write the program to get the 'x' value from P1 and send x^2 to P2, continuously

Lookup table



1. Data Declare (output) array
2. Main Code ↴ array access করে output নেয়া.

config as input

```
ORG 00H
Mov P1, #0FFH
Mov DPTR, #200H
Mov A, P1
MovC A, @A+DPTR
Mov P2, A
SJMP $ (infinity loop)
```

```
ORG 200H
DATA: DB 0,1,4,9,16,25,36,49,64,81
END
```

* Qus ↴ ROM এরকম? Index addressing mode এর.

27 July 26
C-05

Chapter-6

Arithmetic & logic Instructions & Program

106/617

*

ADD A, Source ; A = A + Source

→ Carry flag

ADDC A, Source ; A = A + Source + CY

Add with carry

```
   3C   E7
   3B   8D
  ----------
   78   74
```

Lower half: 1110 0111 (E7) + 1000 1101 (8D) = 0111 0100, CY = 1 (7 4)

1 → Higher half: 0011 1100 (3C) + 0011 1011 (3B) + 1 = 0111 0111 (7 7)

```
CLR C          ; Carry bit = 0
MOV A, #0E7H
ADD A, #8DH
MOV R6, A
MOV A, #3CH
ADDC A, #3BH
MOV R7, A
```

Subtraction

Syntax: SUBB A, Source ; A = A - Source - CY → carry flag

Subtraction with borrow

Multiplication

Syntax: MUL AB ; A × B must A, B reg ১ value থাকতে হবে।

Multiplication	Operand 1	Operand 2	Result
byte × byte	A	B	B = Higher byte A = low byte

Product: high / lower

EE	70
B	A

Division

DIV AB ; divide A by B, A/B

Division	Numerator	Denominator	Quotient	Reminder
byte/byte	A	B	A	B

Example (200)

a) Write a program to get hex data in the range of 00-FFH from port 1 and convert it to decimal. Save it in R7, R6 and R5

b) Assuming that P1 has a value of FDH for data, analyze program

must for Data input

```
MOV P1,#0FFH  ; config
MOV A, P1     ; Data read
MOV B,#10
DIV AB
MOV R5, B     ; Low digit
MOV B,#10
DIV AB
MOV R6, B     ; Middle digit
MOV R7, A     ; Higher digit
```

2's compliment

```
CPL A      ; 1's compliment
ADD A, #1  ; 2's compliment
```

8 bit

AND operation:

```
ANL A, source
```

```
MOV A, #35H
ANL A, #0FH
```

```
35H   0 0 1 1 0 1 0 1
0FH   0 0 0 0 1 1 1 1
-----------------------
05H   0 0 0 0 0 1 0 1
```

Example

```
ANL A, #7EH
```

```
1 0 1 0 1 1 1 0
0 1 1 1 1 1 1 0
---------------
0 0 1 0 1 1 1 0
```

0 1 1 1 1 1 1 0

* যেকোনো bit কে change করে 0 করে ফেলতে পারে,
↳ কাজ 0 করা

OR operation → 1 করতে পারি

ORL A, source

```
MOV  A, #04H
ORL  A, #68H
```

```
04H   0 0 0 0  0 1 0 0
68H   0 1 1 0  1 0 0 0
-----------------------
6CH   0 1 1 0  1 1 0 0
```

XOR → same same bit 0 হয়
inverse করতে পারি, compliment করতে পারি.

XRL A, source

```
MOV  A, #54H
MOV  A, #78H
```

```
54H   0 1 0 1  0 1 0 0
78H   0 1 1 1  1 0 0 0
-----------------------
      0 0 1 0  1 1 0 0
```

toggle

Example: Read & test P1 to see whether it has the value 45H.
If it does, send 99H to P2; otherwise it stays cleared.

```
        MOV P1, #0FFH
        MOV A, P1
        XRL A, #45H
        JZ  equal        ; Jump if Zero
        MOV P2, #0
        SJMP Exit
equal:  MOV P2, #99H
Exit:
```

30 July 26
C-06

Compare Operation

CJNE A, R1, Target → CY = 0 dest ≥ src
dest src CY = 1 dest < src

// equal

Target: Jc less

// dest > src

less: // dest < src

Exercise: Write a program to read the temperature and test it for the value 75. According to the test results, place the temperature value into the registers indicated by the following.

If T = 75 then A = 75
If T < 75 then R1 = T
If T > 25 then R2 = T

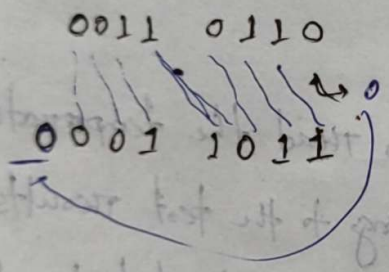
```
MOV P1,#0FFH
MOV A, P1
CJNE A,#75, Not_equal
 MOV A,#75
 SJMP Exit
Not_equal: JNC greater
           MOV R1, A
           SJMP Exit
greater: MOV R2, A
 Exit:
```

Rotate Instruction

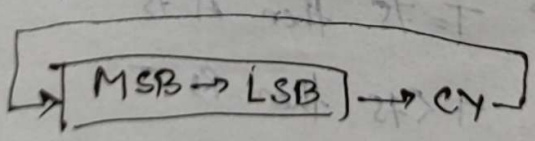
প্রতিটি bit 1 bit/step করে shift করবে, left / right.

যে bit বের হয়ে পরে খালি place এ গিয়ে বসবে।

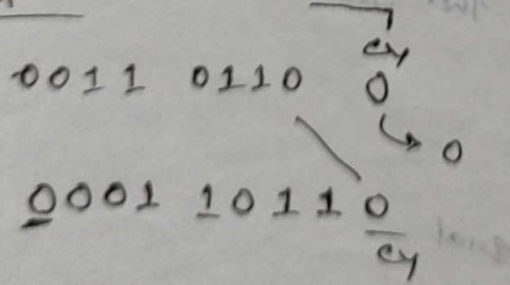
Syntax: RR A



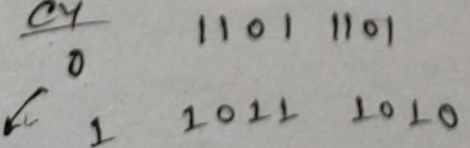
Rotation with Carry



RRC A



RLC A



* Left/Right both case এ Carry bit ব্যবহার করে shift করতে
* RRC তে LSB bit carry তে
* RLC তে MSB bit carry তে

Example: Write a program that finds the number of 1s in a given byte.

```
      MOV  R1, #8
      Mov  A, #58H
      Mov  R0, #0
L:    RRC  A
      JC   Count
      SJMP E
Count: INC  R0
  E:  DJNZ R1, L
```

Example: Write a program to transfer value 41H serially (one bit at a time) via pin P2.1. Put two highs at the at the start and end of the data. Send the byte LSB first.

```
      MOV  R1, #8
      Mov  A, #41H
L:    RRC  A
      MOV  P2.1, C
E:    DJNZ R1, L
```

BCD and ASCII Application

(Binary Coded Decimal)

Single bit unpaced

8

0000 1000

89H

1000 1001

Packed BCD

Page (238/617)

ASCII to BCD

'4' '7'

↓ ↓

34H 37H

↓ |

04H 07H → 04 → 40
07 → 07
47

47H

Steps:

1. Character ला ASCII value ला घेऊन.
2. AND
3. SWAP / Rotation

BCD → ASCII

39H

/ \

'3' 1001

03 Aug, 26
C-7

Timer Programming

Reg. Timer count করতে পারে।
load { initial count value.

0. TMOD load (Reg)
1. load initial count
2. Timer start (TR0, TR1: High (ON), Low (off))
3. Flag check (TF0; TF1)
4. if Flag=1, Timer off/stop
5. Flag clear

Timer 2টা Timer 0, Timer 1

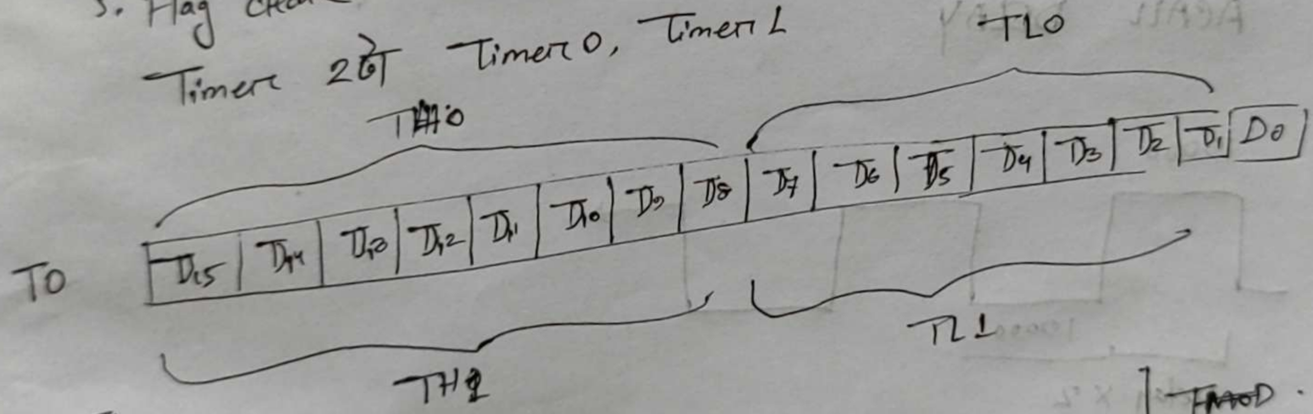
last possible value for reg

FFFF H (16 bit) → 0000 (Rollback) → Flag bit 1 হবে

$2^{16}-1 = 56,$

FFFF
+1
(1) 0000 H (16 bit 0)

1 = Flag bit



B8D2

~~MOV TL0, #D2H~~

load initial count:
MOV TMOD, #01H
MOV TL0, #0D2H
MOV TH0, #0B8H

DELAY:
SETB TR0 (Timer start)
L: JB TF0, Target
SJMP L

Target: CLR TR0
CLR TF0

GATE (control method: 1 = hardware, 0 = software) | C/T (1 = counter, 0 = timer) | M1 | M0 — Timer 1: 0 0 0 0

TMOD

GATE | C/T | M1 | M0 — Timer 0: 0 0 0 1

M1	M0	Mode	operating mode
0	0	0	13 bit timer mode
0	1	1	16 bit timer mode
1	0		
1	1		

Crystal ossilator.

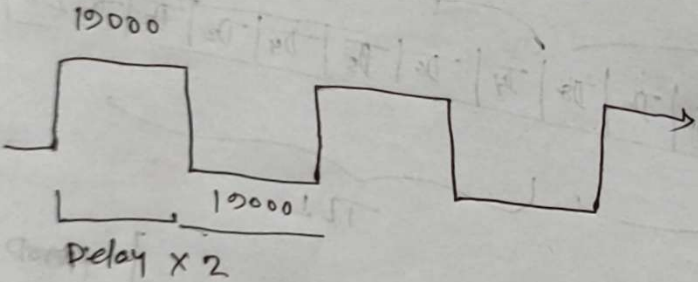
crystal frequency 11.0592 MHz

↳ machine cycle $T = \frac{1}{f} = 1.085$.

I. ~~Del~~ timer count (FFFF − B8D2) + 1

Total delay = 18222 × (1.085 μs)
= 19770.87

```
SETB P1.5
ACALL DELAY
CLR P1.5
ACALL DELAY
```



f | ← → |
T = delay × 2

19770 delay create করতে হলে Timer এর code টা লিখে

Example: 9.4 – 9.9
9.10 – 9.13